

Q: What are four pillars of OOPS

The **four pillars of Object-Oriented Programming (OOP)** are:

1. Encapsulation

- Wrapping **data (fields)** and **methods (functions)** together into a single unit (class) and restricting direct access to the internal details.
- Achieved using **access modifiers** like private, protected, public.
- Example: Hiding a class's variables and exposing only methods to modify them.

```
public class BankAccount
{
    private decimal balance; // hidden

    public void Deposit(decimal amount)
    {
        if (amount > 0) balance += amount;
    }

    public decimal GetBalance() => balance;
}
```

2. Abstraction

- Hiding the **implementation details** and showing only the **essential features** to the user.
- Achieved via **abstract classes** and **interfaces** in C#.
- Example: You know a car's accelerator makes it go faster, but you don't need to know *how* the engine works.

```
public interface IShape
{
    double GetArea();
}
```

3. Inheritance

- The ability to create a new class (child) from an existing class (parent) and **reuse its code**.
- Promotes **code reusability** and a hierarchical relationship.
- Example:

```
public class Animal
{
    public void Eat() => Console.WriteLine("Eating...");
}

public class Dog : Animal
{
    public void Bark() => Console.WriteLine("Barking...");
}
```

4. Polymorphism

- The ability of a single interface (or method) to represent **different forms** or behaviors.
- Two types in C#:
 - **Compile-time (method overloading)**
 - **Run-time (method overriding via virtual/override)**

```
public class Animal
{
    public virtual void Speak() => Console.WriteLine("Animal sound");
}

public class Dog : Animal
{
    public override void Speak() => Console.WriteLine("Bark");
}
```

Q: Difference between protected and internal

1. protected

- **Access Scope** → Accessible **only within the same class** and by **derived classes** (child classes), even if they are in a different assembly.
- **Purpose** → To allow inheritance-based access.
- **Example:**

```
public class BaseClass
{
    protected int value = 10;
}

public class DerivedClass : BaseClass
{
    public void ShowValue()
    {
        Console.WriteLine(value); // ✅ Allowed (inherited member)
    }
}
```

2. internal

- **Access Scope** → Accessible **only within the same assembly** (project), regardless of inheritance.
- **Purpose** → To share code **within the same project** but not outside it.
- **Example:**

```
public class MyClass
{
    internal int value = 20;
}

public class AnotherClass
{
    public void ShowValue()
    {
        var obj = new MyClass();
        Console.WriteLine(obj.value); // ✅ Allowed (same assembly)
    }
}
```



Tip:

You can combine both as protected internal, meaning:

- Accessible by **derived classes** (even in another assembly)
- Accessible by **any class in the same assembly**.

Feature

Access from same class

protected



internal



Access from derived class (same assembly)



Access from derived class (different assembly)



Access from non-derived class in same assembly



Access from different assembly



Types of constructors in C#

1. Default Constructor

- **No parameters**
- Used to initialize fields with default values.
- If you don't define any constructor, C# automatically provides one.

```
csharp
CopyEdit
public class MyClass
{
    public MyClass() // Default Constructor
    {
        Console.WriteLine("Default Constructor Called");
    }
}
```

2. Parameterized Constructor

- Takes parameters to initialize objects with specific values.

```
csharp
CopyEdit
public class MyClass
{
    private int number;
    public MyClass(int num) // Parameterized Constructor
    {
        number = num;
    }
}
```

3. Copy Constructor

- Creates a new object **by copying values** from another object.
- Not built-in; you create it manually.

```
csharp
CopyEdit
public class MyClass
{
    public int Number;
    public MyClass(MyClass obj) // Copy Constructor
    {
        Number = obj.Number;
    }
}
```

4. Static Constructor

- **Called only once** per type, before the first object is created or any static members are accessed.
- Cannot have parameters or access modifiers.

```
csharp
CopyEdit
public class MyClass
{
    static MyClass()
    {
        Console.WriteLine("Static Constructor Called");
    }
}
```

5. Private Constructor

- Used to **restrict instantiation** from outside the class.
- Common in **Singleton** or **Factory** patterns.

```
csharp
CopyEdit
public class MyClass
{
    private MyClass() {}
}
```

6. Chained Constructor (Using this)

- Calls one constructor from another in the **same class**.

```
csharp
CopyEdit
public class MyClass
{
    public MyClass() : this(10) {}

    public MyClass(int num)
    {
        Console.WriteLine("Number: " + num);
    }
}
```

Constructor Type		Parameters	Access Modifier	Purpose
Default	✗		Any	Basic initialization
Parameterized	✓		Any	Pass values at creation
Copy	✓ (object of same type)		Any	Clone object
Static	✗		No modifier	Initialize static data
Private	Optional		private	Restrict instantiation
Chained (this)	Optional		Any	Reuse other constructor logic

Q: Can we instantiate a sealed class

Yes  — a **sealed class** in C# **can be instantiated**, but it **cannot be inherited**

Key Points about sealed classes

- Purpose** → Prevent further inheritance.
- You can still create objects from it directly using the new keyword.
- Commonly used to:
 - Avoid unintended extension of a class
- Optimize runtime (JIT can make certain optimizations on sealed classes)

Tip:

If you want to stop inheritance **only for specific methods**, you can mark those methods as sealed but only if the method is **overridden** in a derived class:

```
public class Base
{
    public virtual void Display() => Console.WriteLine("Base");
}
```

```
csharp

public sealed class Logger
{
    public void Log(string message)
    {
        Console.WriteLine(message);
    }
}

class Program
{
    static void Main()
    {
        // ✓ Allowed
        Logger logger = new Logger();
        logger.Log("Hello from sealed class!");

        // ✗ Not allowed: Cannot derive from sealed class
        // class MyLogger : Logger { } // Compile-time error
    }
}
```

```
public class Derived : Base
{
    public sealed override void Display() => Console.WriteLine("Derived");
}
```

Q: ref vs out

1. ref keyword

- Variable **must** be **initialized** before passing to the method.
- Method **can** read and write the value.
- Often used when you want to **pass a value in** and **possibly change it**.

Example:

```
class Program
{
    static void Modify(ref int number)
    {
        number += 10; // Can read the value
    }

    static void Main()
    {
        int x = 5;    // Must be initialized
        Modify(ref x);
        Console.WriteLine(x); // Output: 15
    }
}
```

2. out keyword

- Variable **does not need** to be initialized before passing.
- Method **must assign** a value before it returns.
- Often used when you want the method to **return multiple values**.

Example:

```
class Program
{
    static void GetValues(out int a, out int b)
    {
        a = 10; // Must assign
        b = 20; // Must assign
    }

    static void Main()
    {
        int x, y; // No need to initialize
        GetValues(out x, out y);
        Console.WriteLine($"{x}, {y}"); // Output: 10, 20
    }
}
```



Tip:

In **C# 7+**, you can use **out var** to declare variables inline:

```
if (int.TryParse("123", out var result)) { Console.WriteLine(result); }
```

Feature

Must be initialized before call

ref

✓ Yes

out

✗ No

Must assign inside method

✗ No

✓ Yes

Can be used to read value

✓ Yes

✗ No (must assign first)

Common use case	Modify existing value	Return multiple values
Q: Delegate in c#		
A delegate in C# is like a type-safe function pointer — it's an object that holds a reference to a method and allows you to call that method through the delegate. It's mainly used for callback methods, event handling, and implementing design patterns like Observer. 1 Basic Definition • Type-safe → The method signature must match the delegate definition. • Can reference static or instance methods. • Can reference multiple methods (multicast delegates). Syntax: csharp CopyEdit // Declare delegate type public delegate void MyDelegate(string message);		
2 Example – Simple Delegate using System; public delegate void Notify(string msg); // Step 1: Declare delegate class Program <pre>{ static void Main() { // Step 2: Create delegate instance pointing to a method Notify notifier = ShowMessage; // Step 3: Invoke delegate notifier("Hello from delegate!"); } static void ShowMessage(string message) { Console.WriteLine(message); } }</pre> Output: Hello from delegate!		
3 Multicast Delegate • A delegate can point to multiple methods and call them in order. public delegate void MyDelegate(); class Program <pre>{ static void Method1() => Console.WriteLine("Method 1"); static void Method2() => Console.WriteLine("Method 2"); static void Main() { MyDelegate d = Method1; d += Method2; // Add another method d(); // Calls Method1 and Method2 } }</pre>		

```
}
```

4 Built-in Delegate Types

Instead of creating custom delegates, C# provides:

- **Action** → for methods that return void
- **Func** → for methods that return a value
- **Predicate** → for methods that return bool

Example:

```
Action<string> action = msg => Console.WriteLine(msg);
```

```
Func<int, int, int> add = (a, b) => a + b;
```

```
Predicate<int> isEven = n => n % 2 == 0;
```

5 Delegates vs Events

- **Delegate** → Points to a method and can be called directly.
- **Event** → Built on delegates but provides extra safety (only the class that declares the event can invoke it).

Feature	Delegate
Type-safe method pointer	✓
Holds multiple methods (multicast)	✓
Used for callbacks/events	✓
Must match method signature	✓



Q: IEnumerable vs IQueryable

- IEnumerable executes in-memory, LINQ-to-Objects
- IQueryable builds an expression tree for remote execution (e.g., DB)
- IQueryable allows deferred execution and server-side filtering

Why: Misuse here leads to huge performance hits.

Q: What are common mistakes when using async/await in .NET?

A:

- Forgetting await leads to unhandled exceptions
- Blocking with .Result or .Wait() can cause deadlocks
- Mixing async and sync code inappropriately

Why: Async misuse can cripple performance.

Q: How would you secure a Web API in .NET 6?

A:

- Use JWT Bearer authentication with AddAuthentication and AddJwtBearer
- Validate tokens with middleware
- Use HTTPS only, CORS rules, rate limiting
- Consider API gateway for external exposure

Why: API security is often overlooked until production.

Q: How do you handle transactions in .NET with EF Core?

A:

```
using var transaction = await context.Database.BeginTransactionAsync();
```

```
try
```

```
{
```

```
    // operations
```

```
    await context.SaveChangesAsync();
```

```
    await transaction.CommitAsync();
```

```
}
```

```
catch
```

```
{
```

```
    await transaction.RollbackAsync();
```

```
}
```

Why: Seniors should know how to ensure data integrity.

Q: Give an example of using a window function in SQL Server.

A:
sql
CopyEdit
SELECT
 EmployeeId,
 Salary,
 RANK() OVER (ORDER BY Salary DESC) AS SalaryRank
FROM Employees;

Why: Tests knowledge beyond basic CRUD.

Q: How would you identify and fix slow queries in SQL Server?

A:

- Use SET STATISTICS IO/TIME or SQL Profiler to find bottlenecks
- Check execution plan for table scans, missing indexes
- Add appropriate indexes (clustered, non-clustered, covering)
- Avoid SELECT *, use specific columns
- Consider query rewriting

Why: Full-stack devs must also be DB performance-aware.

Entity Framework

Q: What is the difference between AsNoTracking() and tracked queries in EF Core?

A:

- Tracked queries keep an entity in the context, so changes are auto-detected
- AsNoTracking() improves performance for read-only queries because it skips change tracking

Why: Senior devs should optimize EF queries knowingly.

MVC 5

Q: What are some major differences between MVC 5 and ASP.NET Core MVC?

A:

- Core MVC is cross-platform, MVC 5 is Windows/IIS only
- No web.config in Core, uses appsettings.json
- Core uses built-in DI, MVC 5 relied on third-party
- Core merges MVC, Web API, and Razor Pages under one umbrella
- Middleware replaces HTTP Modules/Handlers

Why: Ensures they can adapt from older MVC to Core.

Q: How does model binding work in MVC, and how do you handle complex types?

A:

- Model binding maps incoming request data (form, query, route, JSON) to method parameters or model objects
- Complex types require property names to match incoming keys
- For custom handling, implement IModelBinder

Why: Checks understanding of request-to-code data flow.

Q: What are action filters in MVC and when would you use them?

A:

- Filters let you run code before/after actions (OnActionExecuting, OnActionExecuted)
- Types: Authorization, Action, Result, Exception
- Examples: logging, caching, authentication checks

Why: Filters are critical for cross-cutting concerns.

.NET
Q: What are the key differences between .NET Framework, .NET Core, and .NET 6+? When would you still choose .NET Framework? A: • .NET Framework: Windows-only, legacy enterprise apps, WebForms/WCF support. • .NET Core / .NET 6+: Cross-platform, better performance, unified runtime, minimal APIs. • Choose .NET Framework only if you need unsupported tech (e.g., WebForms, old WCF) and rewriting is not feasible. Why: Shows architectural decision-making and migration awareness.
Q: How would you structure a large .NET 6 application for maintainability? A: • Layered or Clean Architecture: Domain Layer, Application Layer, Infrastructure Layer, Presentation Layer. • Use dependency injection for loose coupling. • Separate DTOs from domain models. • Apply SOLID principles. Why: Reveals their approach to scaling codebases.
Q: Explain the difference between <u>transient</u>, <u>scoped</u>, and <u>singleton</u> lifetimes in .NET DI. A: • Transient: New instance every time it's requested. • Scoped: One instance per request. • Singleton: One instance for the entire app lifetime. Why: Misunderstanding lifetimes often leads to memory leaks or data corruption.
Q: What is the difference between <u>IEnumerable</u>, <u>IQueryable</u>, and <u>List<T></u> in LINQ? A: • IEnumerable: In-memory iteration, LINQ-to-Objects. • IQueryable: Query translated to remote source (e.g., SQL) before execution. • List<T>: A concrete collection stored in memory. Why: Key to writing efficient data queries.
Q: What's the difference between <u>ref</u>, <u>out</u>, and <u>in</u> parameters in C#? A: • ref: Pass by reference, must be initialized before call. • out: Pass by reference, must be assigned in the called method. • in: Pass by reference, but read-only inside the method. Why: Tests detailed language knowledge.
Q: What are <u>Middleware components</u> in ASP.NET Core? Give a real-world example. A: • Middleware processes requests/responses in a pipeline. • Example: Logging middleware, authentication middleware, exception handling middleware. Why: Checks if they understand request flow.
Q: How would you secure an ASP.NET Core Web API? A: • Use HTTPS, JWT bearer tokens, role-based authorization. • Implement input validation & data sanitization. • Configure CORS for specific domains. • Rate limiting for public APIs. Why: Security knowledge is critical for senior roles.

Q: How do you implement global exception handling in ASP.NET Core?

A:

- Use UseExceptionHandler middleware or a custom middleware.
- Log exceptions (e.g., Serilog, NLog).
- Return proper HTTP status codes with error details.

Why: Separates production-ready devs from hobbyists.

Q: What are some key differences or new features in .NET 6 compared to earlier .NET Core versions?

A:

- Minimal APIs for lightweight endpoints without full MVC
- Improved performance (e.g., file I/O, JSON serialization)
- Hot reload for faster dev cycles
- Cross-platform and unified platform (no separate .NET Framework/Core/Mono)
- Built-in support for nullable reference types
- File-scoped namespaces for cleaner syntax

Why: Tests awareness of modern .NET practices and if they've kept up with the platform.

Q: How is dependency injection implemented in .NET 6 and why is it important?

A:

- Built-in DI container available via IServiceCollection and IServiceProvider
- Register services in Program.cs using AddSingleton, AddScoped, AddTransient
- DI promotes loose coupling, easier testing, and maintainable code

Why: Confirms they know DI fundamentals and not just copy-paste registrations.

Q: Explain the middleware pipeline in ASP.NET Core. Give an example of a custom middleware.

A:

- Middleware is executed in the order it's registered in Program.cs/Startup.cs
- Each middleware can process request, modify it, call next() to pass to the next middleware, and process the response

- Example custom middleware:

```
public class RequestLoggingMiddleware
{
    private readonly RequestDelegate _next;
    public RequestLoggingMiddleware(RequestDelegate next) => _next = next;

    public async Task Invoke(HttpContext context)
    {
        Console.WriteLine($"Request: {context.Request.Path}");
        await _next(context);
    }
}
```

```
}
```

Why: Middleware is key to ASP.NET Core's request processing model.

SQL Server

Q: How would you troubleshoot a slow SQL query in production?

A:

- Check execution plan.
- Look for table scans → add proper indexes.
- Analyze missing index suggestions.
- Avoid SELECT *.
- Optimize joins and subqueries.

Why: Senior devs must tune queries themselves.

Q: What is the difference between a clustered and a non-clustered index?

A:

- Clustered: Sorts and stores data physically; only one per table.
- Non-clustered: Separate structure that points to data; can have many per table.

Why: Essential for performance tuning.

Frontend Integration

Q: How would you call a .NET API from JavaScript securely?

A:

- Use fetch or Axios over HTTPS.
- Pass JWT in Authorization header.
- Avoid exposing secrets in client-side code.

Why: Tests understanding of API + frontend interaction.

Q: What's your approach to handling CORS in a real-world app?

A:

- Restrict allowed origins, methods, and headers.
- Avoid using AllowAnyOrigin in production.
- Use environment-based CORS policies.

Why: Senior devs think about security by default.

Q: How would you handle 1 million concurrent users hitting your API?

A:

- Use async/await for non-blocking I/O.
- Add caching (e.g., Redis) for frequent queries.
- Horizontal scaling with load balancers.
- Use database connection pooling.
- Consider message queues for heavy processing.

Why: Evaluates scalability thinking.

Q: How would you improve EF Core performance for a reporting query?

A:

- Use AsNoTracking() for read-only queries.
- Project to DTO instead of returning entity.
- Use compiled queries if repeated.
- Avoid lazy loading in large datasets.

Why: Shows optimization mindset.

Problem Solving
Q: You deployed a new API version, and users report occasional timeouts. How do you debug it?
A: • Check logs & Application Insights traces. • Monitor server metrics (CPU, RAM, network). • Identify slow queries in DB. • Simulate load with a stress test. • Roll back if necessary.
Why: Tests crisis-handling ability.